Verification and Validation Report: Flow

Team 9, min-cut
Ethan Patterson
Hussain Muhammed
Jeffrey Doan
Kevin Zhu
Chengze Zhao

March 10, 2026

1 Revision History

Date	Version	Notes
09/03/2026	1.0	Initial Release

2 Symbols, Abbreviations and Acronyms

N/A

Contents

1 Revision History **i**

2 Symbols, Abbreviations and Acronyms **ii**

3 Functional Requirements Evaluation **1**
3.1 Functional User Testing 1
3.2 User Feedback 2
3.3 Summary 3

4 Nonfunctional Requirements Evaluation **3**
4.1 Usability 3
4.2 Performance 6

5 Comparison to Existing Implementation **6**
5.1 Pen and Paper 6
5.2 Tablet-Based Note Taking 7
5.3 Traditional Text Editors 7
5.4 Mouse-Driven Diagram Editors 7
5.5 Text-Based Diagram Languages 8
5.6 Document Markup Languages 8
5.7 Summary 8

6 Unit Testing **9**
6.1 File Interface Module (M3) 9
6.2 Command Parser Module (M6) 9
6.3 Geometry State Module (M11) 9
6.4 Geometry State Mutator Module (M12) 9
6.5 Undo Redo Module (M13) 9

7 Changes Due to Testing **10**
7.1 Feedback Summary 10
7.1.1 Professor Feedback 10
7.1.2 Teaching Assistant Feedback 11
7.1.3 User Testing Feedback 11
7.2 Future Changes To Be Implemented in Response to Feedback 12
7.2.1 Architecture Decision: Infinite Canvas vs. Separate Pages 12

7.2.2 PDF Export Implementation 12
7.2.3 Work Distribution Improvements 12
7.2.4 User Testing Feedback Integration 13
7.3 Changes Already Implemented in Response to Feedback 13
7.4 Impact on Project Quality 14

8 Automated Testing **14**

9 Trace to Requirements **15**

10 Trace to Modules **17**

11 Code Coverage Metrics **17**

List of Tables

1 Summary of Functional User Testing Results 2
2 Likelihood of Using Flow Over Existing Tools (1–5) 5
3 Traceability Between Requirements and Test Cases 16
4 Traceability Between Requirements and Test Cases 20

List of Figures

1 Mode Clarity Rating (1–5) 4
2 Deal breaker to Adopting Flow Full-Time 6

3 Functional Requirements Evaluation

3.1 Functional User Testing

User testing was conducted to evaluate whether the implemented editing and canvas manipulation functionality satisfies the functional requirements specified in the SRS. Participants were asked to perform a series of diagram creation tasks using Flow for the first time. These tasks involved inserting shapes, connecting elements with lines, and positioning objects on the canvas using the keyboard-driven interaction model.

Two participants completed the testing procedure. Each participant performed the same task twice in order to observe how performance changed after gaining familiarity with the interface.

The first participant completed the task in approximately 3 minutes and 7 seconds during the first run and made roughly ten errors. Most of these errors occurred because the participant was initially unaware that they could switch between line mode and anchor-line mode in order to continue the same line between shapes. After becoming familiar with this behavior, the participant completed the second run in approximately 1 minute and 49 seconds with only one or two minor errors.

The second participant required approximately 8 minutes and 46 seconds to complete the first run and approximately 6 minutes and 23 seconds for the second run. During these runs, the participant performed several accidental operations that required undo actions and occasionally forgot how to execute specific commands, such as adding text to a shape or switching interaction modes.

Despite these initial usability challenges, both participants were able to successfully complete the required diagram creation tasks. The reduction in completion time between runs suggests that users become more efficient after a short familiarization period with the keyboard-driven workflow.

Overall, the testing results indicate that the implemented editing and canvas manipulation functionality operates as intended. Users were able to create and manipulate diagram elements and complete the assigned tasks success-

Participant	Run	Completion Time	Errors / Issues
User 1	Run 1	3:07	∼10 errors
User 1	Run 2	1:49	1–2 errors
User 2	Run 1	8:46	2 undo, 3 command issues
User 2	Run 2	6:23	3 undo, 3 command issues

Table 1: Summary of Functional User Testing Results

fully. However, the tests also reveal that the keyboard-driven interface introduces a learning curve for first-time users, particularly when remembering available commands and interaction modes.

3.2 User Feedback

After completing the testing tasks, participants were asked a series of questions about their experience using Flow. The goal of these questions was to evaluate how intuitive the system felt to first-time users and to identify areas where the interaction design could be improved.

Participants highlighted the line creation mode as one of the most effective features of the system. In particular, the automatic snapping behavior and the ability to extend connections between shapes without manually selecting connection points were viewed as highly convenient when constructing diagram structures. Users also noted that positioning elements on the canvas felt intuitive compared to text-based diagram tools.

Several usability challenges were also identified during testing. Both users reported difficulty remembering all available keyboard shortcuts during their initial interaction with the system. One participant suggested that a reference sheet or command palette could help users discover available commands more easily.

Participants also suggested several potential feature improvements, including support for inserting custom icons, automatic linking of shapes during diagram creation, and the ability to select all elements on the canvas. These suggestions indicate potential directions for future development that could

further improve the usability and flexibility of the system.

Overall, the feedback suggests that Flow's keyboard-driven interaction model is powerful for diagram construction, but additional discoverability features may be beneficial to reduce the learning curve for new users.

3.3 Summary

The functional evaluation demonstrates that the core diagram creation and editing capabilities of Flow operate as intended. All participants were able to successfully complete the assigned tasks using the implemented features, confirming that the system satisfies the primary functional requirements related to canvas editing and diagram manipulation.

The testing results also show that user efficiency improves noticeably after a short familiarization period. Completion times decreased between the first and second runs for both participants, suggesting that the keyboard-based interaction model becomes more efficient once users become comfortable with the available commands.

While the core functionality performed correctly, the evaluation also identified usability challenges for first-time users. In particular, participants experienced difficulty remembering keyboard shortcuts and distinguishing between certain interaction modes. These observations suggest that improved command discoverability and documentation could further enhance the user experience.

Overall, the implemented system successfully meets the functional goals outlined in the SRS while also revealing opportunities for future usability improvements.

4 Nonfunctional Requirements Evaluation

4.1 Usability

NF217 was the primary requirement evaluated in this section, focusing on visual consistency and an intuitive UI layout. To evaluate this, a usability study was done where participants interacted with Flow while completing diagram

creation tasks. After completing the tasks, participants filled out a questionnaire evaluating aspects of the interface such as mode clarity, intuitiveness, and perceived ease of learning. Participants also provided background information about their existing note-taking workflows to contextualize their responses. The study involved two participants, both of whom were familiar with digital note-taking tools and regularly take notes on tablets or other digital platforms.

Participants were asked to rate how clearly the interface communicated the current editing mode (e.g., Insert, Normal, Visual) on a scale from 1 to 5. As shown in Figure 1, the average rating was 4.5/5, indicating that users were generally able to identify the current mode and understand the visual cues provided by the interface. Participants also evaluated the overall intuitiveness of the system. Both users reported that the system felt intuitive once commands were demonstrated, suggesting that the layout and interaction model are conceptually understandable despite being command-driven.

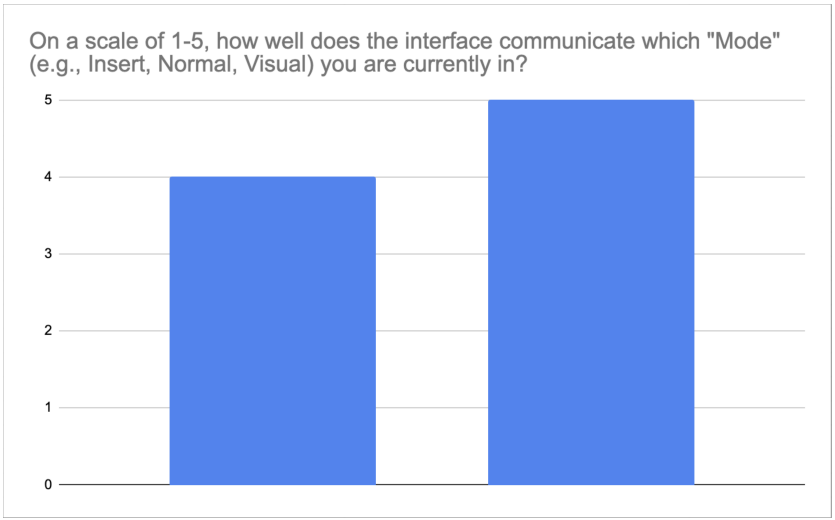

Figure 1: Mode Clarity Rating (1–5)

However, responses related to switching to the app were more moderate. When asked how likely they would be to use Flow over their existing note-taking tools, participants gave responses around the 2-3 out of 5 range. More

specifically, Table 4.1 below describes the average ratings for adoping flow over other methods of note-taking. This indicates that while the interface was understandable, they were not immediately inclined to replace their current workflows. Similarly, when asked about potential barriers to adopting Flow full-time, responses were split between "lack of features or maturity" and "too difficult to learn.", seen in the pie graph in Figure 2. These responses suggest that while the interface is relatively clear and consistent, the learning curve and perceived feature completeness may affect early adoption.

Overall, the results indicate that Flow successfully communicates its modal interface and maintains a consistent layout, supporting the usability requirement. Participants were able to understand the system's structure and reported that the interaction model felt intuitive once commands were learned. However, the results also highlighted areas for improvement, including reducing the perceived learning barrier and expanding functionality. Based on these observations, the requirement for visual consistency and intuitive UI layout is considered largely satisfied, though further refinement may improve user adoption and long-term usability.

Tool	**Avg Rating**
Pen & Paper	2.5
Tablet	2.5
Online Docs	3
Online Notes	3
Online Drawing	3

Table 2: Likelihood of Using Flow Over Existing Tools (1–5)

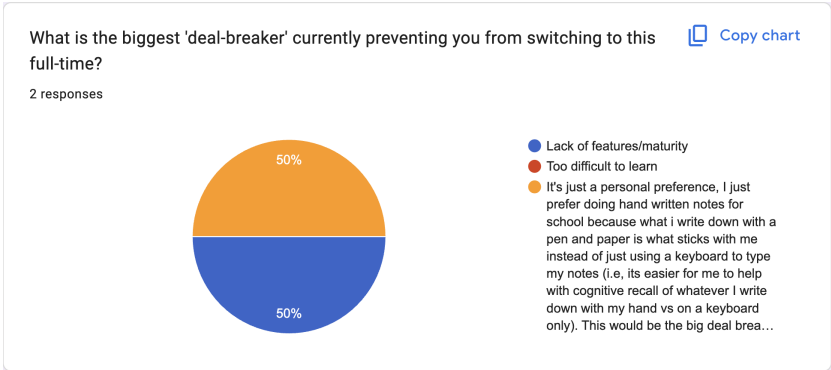

Figure 2: Deal breaker to Adopting Flow Full-Time

4.2 Performance

The performance was determined by using electrons clock to track the time when the program gets the input and the time after then new frame has been drawn.
It was found to take around 0.1 to 0.2 ms to insert an empty circle and around 33 ms to insert a text box into an empty document.
These numbers did not notably change with 100+ shapes in the document but at 300+ shapes the latency increased by around .2 ms for circles and around 3 ms for text boxes.

# Shapes	Circle	Text Box
0	0.1ms	33.2ms
100	0.2ms	31.6ms
200	0.2ms	32.0ms
300	0.3ms	33.5ms

5 Comparison to Existing Implementation

5.1 Pen and Paper

Pen-and-paper note taking allows quick sketches and diagrams, but it lacks digital advantages such as editing, sharing, and organization. Notes cannot easily be rearranged or modified without rewriting large portions.

The Flow prototype improves on this by allowing users to freely edit diagrams and text on a digital canvas. Shapes and text boxes can be moved, resized, and modified without recreating the content. This allows iterative editing that is not possible with traditional handwritten notes.

5.2 Tablet-Based Note Taking

Tablet systems such as the iPad with Apple Pencil allow freehand diagramming and handwritten notes. While flexible, these solutions require specialized hardware and often rely on handwriting input, which may be slower than typing for many users.

Flow instead targets keyboard-based workflows and runs on standard desktop computers. The prototype demonstrates keyboard-driven editing and diagram manipulation, allowing users to create structured notes and diagrams without requiring additional hardware.

5.3 Traditional Text Editors

Applications such as Microsoft Word or Google Docs support structured text editing but provide limited support for integrated diagram creation. Diagrams often require external tools or inserted images.

The Flow prototype integrates diagram creation directly into the editing workspace. Users can insert geometric shapes and connect them with arrows directly on the canvas while simultaneously editing text content. This removes the need to switch between separate diagramming tools.

5.4 Mouse-Driven Diagram Editors

Tools such as Draw.io or OneNote allow diagram creation but rely heavily on mouse-based interaction and menu navigation. Creating shapes or connections often requires multiple UI interactions.

Flow differs by emphasizing keyboard-driven interaction. The current prototype demonstrates Vim-style navigation and keyboard commands that allow users to manipulate shapes, text boxes, and diagram elements without relying on extensive mouse interaction.

5.5 Text-Based Diagram Languages

Diagram languages such as Mermaid or PlantUML allow diagrams to be described using text. While powerful, they require a separate compilation or rendering step to visualize the diagram.

Flow eliminates this separation by providing direct manipulation of diagram elements on the canvas. The rendering engine updates the diagram in real time as the user edits elements, avoiding the write–compile–preview workflow required by text-based diagram languages.

5.6 Document Markup Languages

Document preparation systems such as LaTeX or Typst support complex document layouts and diagram generation. However, these systems rely on writing structured markup code and are typically designed for page-based documents rather than an interactive canvas.

Flow instead uses a canvas-based editing environment where diagram elements and text boxes can be freely positioned. The prototype supports interactive editing of shapes and text elements, which allows users to experiment with diagram layouts more quickly than code-based document systems.

5.7 Summary

The Flow prototype demonstrates that a keyboard-driven, canvas-based note-taking system can combine advantages from multiple existing approaches. Compared to existing tools, the system:

- Integrates text editing and diagram creation in a single interface
- Supports keyboard-driven workflows inspired by Vim
- Allows direct manipulation of diagram elements without compilation or external tools
- Provides a flexible canvas for positioning text and diagrams

Although the current implementation does not yet include persistent storage, exporting, or collaboration features, the prototype validates the feasibility

of the core interaction model and demonstrates improvements over several existing approaches.

6 Unit Testing

6.1 File Interface Module (M3)

This functionality is not implemented yet and therefore not tested yet.

6.2 Command Parser Module (M6)

All tests passing. Sample test cases outlined in the VnV plan were implemented, including others.
Implementations in ‘test/unit/core/commands/CommandParser.spec.ts‘ (on the ‘develop‘ branch)

6.3 Geometry State Module (M11)

All tests passing. Sample test cases outlined in the VnV plan were implemented, including others.
Implementations in ‘test/unit/core/document/Document.spec.ts‘ (on the ‘develop‘ branch)

6.4 Geometry State Mutator Module (M12)

All tests passing. Sample test cases outlined in the VnV plan were implemented, including others.
Implementations in ‘test/unit/core/commands/ManipulationCommands.spec.ts‘ (on the ‘develop‘ branch)

6.5 Undo Redo Module (M13)

All tests passing. Sample test cases outlined in the VnV plan were implemented, including others.
Implementations in ‘test/unit/core/document/History.spec.ts‘ (on the ‘develop‘ branch)

7 Changes Due to Testing

Following the Rev 0 demonstration, comprehensive feedback was received from both the course professor and teaching assistant. This feedback directly influenced the project's direction and implementation priorities, resulting in significant improvements to the system's design, usability planning, and development approach. Additionally, usability tests were conducted with outside users to gather real-world insights and further improve the product based on actual user interactions.

7.1 Feedback Summary

7.1.1 Professor Feedback

The professor provided detailed feedback during Rev 0 highlighting both strengths and areas for improvement:

- **Positive Recognition**: The demonstration effectively showcased the classroom note-taking concept, representing a significant improvement over the Proof of Concept demo. The project vision was described as "exciting to see come together."
- **Productivity Report Enhancement**: The demo plan section in the productivity report required more detailed planning and structure, similar to the presentation format.
- **Usability Testing Planning**: Emphasized the critical importance of comprehensive usability testing, requiring detailed planning addressing who, when, where, what, and how aspects. Specific guidance included:
  - **What**: Define specific tasks for test subjects to perform
  - **How**: Establish clear metrics for measuring usability, likely focusing on user interviews
  - **Planning**: Questions should be prepared in advance
- **Feature Prioritization**: Recommended implementing PDF export functionality as a high priority, while "flow" file saving could be deprioritized. Additionally, The team must decide between infinite canvas or separate pages for adding additional slides/ diagrams.

7.1.2 Teaching Assistant Feedback

The TA provided additional insights complementing the professor's assessment:

- **Demo Quality**: Described the demonstration as "smooth and impressive" with much clearer project vision compared to the PoC demo.
- **Usability Testing Considerations**: Emphasized the importance of considering users' prerequisite knowledge (such as Vim proficiency) and how this affects test result interpretation.
- **Feature Prioritization**: Reinforced the need to prioritize features critical to the final demonstration.

7.1.3 User Testing Feedback

We conducted a usability test with two individuals that have a technical background, but is not part of the development team. The users were asked to perform a series of tasks using the current prototype, and their interactions were observed and recorded. The users provided the following feedback:

- **Auto Linking Mode**: The users suggested that an auto-linking mode where the system automatically creates a line connection between two shapes upon object creation. New shapes will automatically connect to the most recently created shape. This would reduce the number of steps required to create a diagram and make it easier.
- **Select All**: The users requested a "select all" feature that allows users to select all objects and elements on the canvas with a single command. A simple feature that would improve the quality of life for users.
- **Custom Icons**: The users suggest adding custom icons such as different circuit symbols and having keybinds such as "ii" to insert an icon and have fuzzy find a list of available icons.
- **Anchor points**: The users suggest adding custom anchor points when adding a custom icon. This would allow users to specify where the connection points of their icons are, making it easier to create diagrams that require custom icons.

- **End Remarks and Comparisons**: The user expressed that the current prototype would be a great product and made a comparison to other circuit makers in typst which they were not quite fond of. They expressed that the product would be great at drawing circuits if provided with a few more features listed above.

7.2 Future Changes To Be Implemented in Response to Feedback

7.2.1 Architecture Decision: Infinite Canvas vs. Separate Pages

Following the professor's recommendation, the team conducted an architectural analysis and is still deciding on a solution. This decision will be based on:

- While implementing both solutions would be ideal, we have considered time constraints as an issue, and would prioritize implementing one solution over the other.
- Better alignment with the classroom note-taking workflow. The infinite canvas would allow users to freely position and organize their notes; however, the separate pages will help with organization and navigation.
- In the end, the team decided that we will provide support for uploading and viewing a PDF slide, whether that be through an infinite canvas, multiple pages, or both.

7.2.2 PDF Export Implementation

Prioritizing the professor's recommendation, PDF export functionality will be implemented as a core feature, allowing users to export their diagrams in a widely compatible format suitable for sharing and archiving.

7.2.3 Work Distribution Improvements

Addressing the concern about uneven workload distribution, the team implemented several measures:

- Clear task assignment and rotation across different project areas

- Regular code review processes to ensure knowledge sharing
- Documentation improvements to facilitate team-wide understanding

7.2.4 User Testing Feedback Integration

Earlier usability testing produced several feature suggestions from technical users outside the development team. To address these items, we took the following steps:

- Prototyping an "auto-linking" mode: new shapes automatically connect to the most recently created object, reducing steps to draw diagrams.
- Adding a global `select all` command, enabling users to highlight every canvas element with one keystroke.
- Looking into extending the icon insertion system to support custom symbols with a fuzzy-find picker and permitted definition of custom anchor points for those icons to control connection locations.
- Implementing miscellaneous quality-of-life tweaks (keyboard hints, clearer messages) based on the user's general remarks.
- Considering additional suggestions (circuit symbol libraries, advanced anchoring) in the project backlog for future iterations.

7.3 Changes Already Implemented in Response to Feedback

The team has already implemented several changes in response to feedback/testing from internal usage and the POC demo:

- **Keybinding Layout**: We have put more thought and effort into the keybinding layout, ensuring that it is more intuitive and easier to learn for users familiar with Vim-style keybindings. This was well recieved during user testing and has improved the overall usability of the prototype.

- **Visual Indicators**: We have added visual indicators for the different modes users are able to be in (normal, insert, visual). Now the cursor will change based on the mode, signaling to the user which mode they are in. This has helped users understand the current state of the application and has reduced confusion during interactions.

- **Line Anchor Points**: A small but impactful change was the addition of anchor points for line connections. This allows users to connect to created lines instead of just shapes, enabling more complex diagrams and improving the flexibility of the system. This was a direct result of internal testing and POC feedback.

7.4 Impact on Project Quality

The feedback-driven changes significantly enhanced the project's overall quality:

- **Enhanced Architecture**: The infinite canvas and separate pages decision provides a more intuitive and flexible user experience aligned with the target workflow.

- **Increased Robustness**: PDF export functionality addresses a critical user requirement for document sharing and archiving.

- **Better Team Dynamics**: Improved work distribution ensures all team members develop relevant skills and contribute meaningfully to the project.

The verification and validation process, through structured feedback collection and responsive implementation, has ensured that the Flow project evolves based on expert guidance and user needs, resulting in a higher-quality final product that better serves its intended educational purpose.

8 Automated Testing

As outlined in the VnV Plan, we implemented automated testing using the `Jest` framework. We also projected the use of `Playwright` in the original plan, but as of this report, we have not used it for any testing. Due to the rapid development and changes made to the app, we have not invested the

time to create end-to-end tests at this point that may be more brittle than unit tests and slow down development speed. Additionally, usage/integration of `Coveralls` was discussed in the plan, but has not been added at this point due to time constraints and its relatively low priority. We have been able to use Jest to check code coverage.
Automated tests are run via GitHub Actions and run on every new PR.

9 Trace to Requirements

This section provides traceability between the requirements defined in the Software Requirements Specification (SRS) and the verification activities performed during testing. Each requirement is linked to one or more test cases described in the Verification and Validation Plan. The results of the tests demonstrate whether the implementation satisfies the specified requirements.
Functional requirements are validated primarily through automated system tests that simulate user interactions with the application. These tests verify core functionality such as creating notes, manipulating diagram elements, and performing file management operations.
Non-functional requirements are evaluated through a combination of manual tests, usability evaluation, and performance measurements. These tests assess system responsiveness, usability, cross-platform compatibility, and data integrity.
Table 4 summarizes the mapping between SRS requirements and the test cases used to verify them.
The results indicate that all implemented requirements were successfully verified through the corresponding tests for features that have been implemented so far. No critical failures were observed during the verification process, demonstrating that the implemented systemsatisfies the functional and non-functional requirements specified in the SRS.

Req ID	Requirement Description	Test Case	Result
F211	The system shall allow users to create and edit notes.	test-F211	Pass
F214	The system shall allow users to insert and manipulate shapes on the canvas.	test-F214	Pass
F217	The system shall allow users to save and reopen notes.	test-F217	Not Implemented Yet
F243	The system shall allow users to export notes as PDF files.	test-F243	Not Implemented Yet
NF211	The system shall provide low-latency input and rendering during editing operations.	test-NF211	Pass
NF218	The system shall maintain acceptable performance when handling a large number of shapes or objects.	test-NF218	Pass
NF213	The system shall support execution on Windows, macOS, and Linux platforms.	test-NF213	Pass
NF214	The system shall provide accessible keyboard shortcuts for editing operations.	test-NF214	Pass
NF217	The system shall provide an intuitive and consistent user interface.	test-NF217	Pass
NF212	The system shall maintain local data persistence and autosave functionality.	test-NF212	Not Implemented Yet
NF215	The system shall maintain file integrity during save and export operations.	test-NF215	Not Implemented Yet
NF216	The system shall not perform background network communication.	test-NF216	Pass

Table 3: Traceability Between Requirements and Test Cases

10 Trace to Modules

This section relates between the modules defined in the Module Interface Specification (MIS) and the tests being performed. Every test is related to one or modules in defined in the MIS. The results of the tests show that the modules are properly implemented. Table 4 summarizes the mapping between MIS modules and the test cases used to verify them.

11 Code Coverage Metrics

Line coverage: 41.98%
Branch coverage: 29.54%

References

Test Case	Test Requirement Description	Related Modules	Test Status
test-F211	The system shall allow users to create and edit notes.	Display Interface Module, , Input Interface Module, , Geometry State Parser Module, , Geometry State Converter Module, , Commands Parser Module, , Mode Commands Module, , Shape Interface Module, , Text Buffer Module , Geometry State Module , Geometry State Mutator Module	Pass
test-F214	The system shall allow users to insert and manipulate shapes on the canvas.	Display Interface Module , Input Interface Module , Geometry State Parser Module , Geometry State Converter Module , Commands Parser Module , Mode Commands Module , Shape Interface Module , Geometry State Module , Geometry State Mutator Module	Pass
test-F217	The system shall allow users to save and reopen notes.	Input Interface Module , File Interface Module , Commands Parser Module , Mode Commands Module , User Persistence Module	Not Implemented Yet
test-F243	The system shall allow users to export notes as PDF files.	Input Interface Module , File Interface Module	Not Implemented Yet
test-NF211	The system shall provide low-latency input and rendering during editing operations.	Display Interface Module , Input Interface Module , Commands Parser Module , Mode Commands Module	Pass

test-NF218	The system shall maintain acceptable performance when handling a large number of shapes or objects.	Display Interface Module , Input Interface Module , Commands Parser Module , Mode Commands Module	Pass
test-NF213	The system shall support execution on Windows, macOS, and Linux platforms.	N/A	Pass
test-NF214	The system shall provide accessible keyboard shortcuts for editing operations.	Input Interface Module , Commands Parser Module , Mode Commands Module , User Preference Module	Pass
test-NF217	The system shall provide an intuitive and consistent user interface.	Display Interface Module , User Preference Module	Pass
test-NF212	The system shall maintain local data persistence and autosave functionality.	File Interface Module , User Persistence Module	Not Implemented Yet

test-NF215	The system shall maintain file integrity during save and export operations.	File Interface Module , User Persistence Module	Not Implemented Yet
test-NF216	The system shall not perform background network communication.	N/A	Pass

Table 4: Traceability Between Requirements and Test Cases

Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Reflection.
The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process. Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."
Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

    - Ethan: One thing that went well during this deliverable was that it motivated the team to conduct testing in preparation for writing it. This included conducting user testing and writing more

unit tests. The usability testing was especially helpful and fun, as it was the first time we got to see how people outside of our team interacted with our project. Overall we got some really good feedback. Additionally this report helped us to organize the features we can prioritize based on user feedback as we move towards the final demo.

- Hussain: In this deliverable, we were finally able to get useful, concrete feedback from other people. This was the first time we had a chance to officially show our work to people outside of our team, and it was very helpful. Although we are also stakeholders in this project, showing other people that are not as close to the project allowed us to get a different perspective on our work, pointing out things that we had overlooked. From this feedback, I look forward to improving our project and making it more user-friendly.
- Jeffrey: In this deliverable, we were able to get a lot of feedback and put them to action. As a team, we were able to finally get a feel for what our users want and how they interact with our product. This was a very important step for us, as it was one of the first times we were able to get concrete feedback from users which helped guide our development. Additionally, we were able to create tests which will be very helpful for us in the future, as it will allow us to catch bugs and ensure that our code is working as expected. Overall, I think this was a very productive deliverable.
- Chengze: One thing that went well during this deliverable was being able to see how new users interacted with Flow. Watching people use the system for the first time helped us understand which parts of the interface were intuitive and which parts were confusing. It was helpful to get feedback from people outside the team because they noticed things that we had overlooked. Overall, this testing gave us a better idea of how users actually experience our product and what we should improve going forward.
- Kevin: One thing that went well writing this deliverable was having a proper plan for what were were going to do for our VnV in our VnV plan. Even if we had to change our tests due to parts of our program being different than planned, having a proper plan let us simply implement tests we can do and work around ones we can't. Implementing tests also gave us a better idea of what

we need to work on next as both the feedback from our usability testing and our testing gave us a better idea of what the project needs.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   - Ethan: One pain point was catching up on testing. We had not written a lot of unit tests before this deliverable, so we had quite a bit of code to cover in a short amount of time. Additionally, the traceability was a bit difficult to work on as our code/module organization has naturally evolved and changed from our original MG. This is something that we will aim to update for the final documentation.
   - Hussain: One pain point that I experienced during this deliverable was the fact that we did not fully consider the unit testing description, when planning on completing the deliverable. Though we successfully made the unit tests for different parts of our project, we did not have a clear plan on how we will categorize them and how we will present them in the report. This caused a bit more last-minute planning and organization, which was a bit stressful.
   - Jeffrey: One of the biggest pain points I experienced during this deliverable was when we were creating tests for our software. I personally create numerous tests for specific features I implemented; however, was not aware of how we were supposed to document these for the VnV report. This caused a lot of last-minute work to figure out how to categorize and present our tests in the report. In the future, I will make sure to have a clear plan for how to document tests in the report, so that I can avoid this kind of last-minute stress.
   - Chengze: One challenge I had during this deliverable was figuring out how to present the user testing results clearly in the report. We collected a lot of feedback and timing data, but it took some time to decide what information was most important to include. To solve this, I focused on linking the testing results back to the functional requirements and organizing them in a clearer structure. This helped make the evaluation section easier to understand and showed how the tests relate to the requirements.

- Kevin: While having unit test written already was very helpful in directing our testing, one pain point of this deliverable was having to work around the changes and undecided parts of our plan. For example, when I was doing the performance testing our plan our test plan only included basic shapes but I decided to also include text boxes as they are also shapes that have spawn latency that can affect the user.

3. Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?

   The parts of this document that stemmed from client/user input are the sections related to usability testing: section 4.1. This is because to properly gauge how the program feels to use we needed input from users who have never used the program before, letting us test learnability and other problems that someone who worked on the program may not find. Other tests were related to the development or functions (unit tests) which did not need as much outside input to properly test.

4. In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV? If there were differences, what changes required the modification in the plan? Why did these changes occur? Would you be able to anticipate these changes in future projects? If there weren't any differences, how was your team able to clearly predict a feasible amount of effort and the right tasks needed to build the evidence that demonstrates the required quality? (It is expected that most teams will have had to deviate from their original VnV Plan.)

   Some changes that required a modification to the plan was how many of the tests were written for modules that were not yet implemented. For example, we had tests relating to file management that were unused as we had not added a proper way to save files so that they can be reopened with all shapes still editable. Additionally, our team decided to do usability testing even though it was originally considered a stretch goal in our VnV plan. This is as during our meeting with the professor he discussed that for our program, usability testing is the most important as it is the testing that shows if our program is of the required quality and if users are willing to use it over other implementations.